

Exercise: OOP in OCaml

Programming Language & Compiler

School of Computer Science, Wuhan University

练习 1 Calculator

目的

- 体会 OCaml 中 Variant 的定义和使用。
- 复习之前的 Tutorial。

题目描述

请使用 OCaml 语言，实现一个简单的算术表达式计算器。

我们使用以下代数数据类型 `expr` 来表示算术表达式，你需要补充乘法和除法的定义：

```
type expr =  
  | Const of int          (* 整数常量 *)  
  | Add of expr * expr    (* 加法运算 *)  
  | Sub of expr * expr    (* 减法运算 *)
```

请你编写一个函数 `evaluate`，用于计算给定表达式 `expr` 的值。函数 `evaluate` 应当满足：

```
val evaluate : expr -> int option
```

该函数返回 `Some v`，表示计算出的整数结果 `v`。该函数返回 `None`，表示计算失败，比如除以零的情况。

样例

```
(* 3 + 2 * 5 *)  
let expr = Add (Const 3, Mul (Const 2, Const 5))  
let result = evaluate expr
```

输出是：

```
(* 3 + 2 * 5 = 13 *)  
val result : int option = Some 13
```

提示

使用递归函数和模式匹配来完成这个任务。

练习 2 Bank Account

目的

- 学习使用 OCaml 中的 `mutable` 和 `ref`。
- 学习使用 OCaml 中的 `exception`。

题目描述

创建一个银行账户系统，使其具备以下功能：

- 账户创建：每个账户必须包含唯一的用户 ID 和所有者姓名。
- 存款：向账户余额中添加一定金额。如果存款金额为负数，抛出 `InvalidDeposit` 异常。
- 取款：从账户余额中取出一定金额：如果余额足够，正常扣除金额并返回。如果余额不足，抛出 `InsufficientFunds` 异常。如果取款金额为负数，抛出 `InvalidWithdrawal` 异常。
- 记录流水：你应该用一个 list 记录下来存取款的历史记录
- 查询余额：获取当前账户余额。

你需要实现一个 `bank_account` 类型。某些值应存储在一个可变字段或引用中，以便更新值。

```
type bank_account = {  
  ...  
}
```

你还需要实现四个函数：存款 `deposit`、取款 `withdraw`、查询余额 `get_balance` 以及输出记录 `print_transactions`

```
val deposit : bank_account -> int -> unit  
val withdraw : bank_account -> int -> unit  
val get_balance : bank_account -> int  
val print_transactions : bank_account -> unit
```

提示

`ref` 和 `mutable` 二选一。使用 `try ... with` 捕获异常并抛出。你可以设计一个变体 `transaction`，由存取款两个类型构造器组成。

练习 3 Map

目的

- 掌握 OCaml 中模块的定义。
- 掌握类型变量的使用。

题目描述

映射 (Map) 是由键值对组成的无序集合, 在这个练习中你需要实现一个自己的 Map:

```
module type Map = sig
  (** [('k, 'v) t] 是一个映射的类型, 它将键 ('k 类型) 绑定到值 ('v 类型)。 *)
  type ('k, 'v) t

  (** [empty] 表示一个不包含任何键的映射。 *)
  val empty : ('k, 'v) t

  (** [insert k v m] 返回一个新的映射, 其中 [k] 绑定到 [v], 并且包含 [m] 中的所有绑定。
      如果 [k] 在 [m] 中已经有绑定, 则新映射中 [k] 的绑定将被 [v] 替代。 *)
  val insert : 'k -> 'v -> ('k, 'v) t -> ('k, 'v) t

  (** [lookup k m] 返回在映射 [m] 中与键 [k] 绑定的值。
      如果 [k] 在 [m] 中没有绑定, 则会引发 [Not_found] 异常。 *)
  val lookup : 'k -> ('k, 'v) t -> 'v

  (** [bindings m] 返回一个关联列表, 该列表包含 [m] 中所有的绑定关系。
      该列表中的键保证是唯一的。 *)
  val bindings : ('k, 'v) t -> ('k * 'v) list
end
```

样例 1

```
let m = MyMap.empty;;
let m = MyMap.insert "a" 1 m;;
let m = MyMap.insert "b" 2 m;;
let m = MyMap.insert "c" 3 m;;
MyMap.lookup "b" m;;
MyMap.lookup "a" m;;
```

```
-: int = 2
-: int = 1
```

样例 2

```
let m = Map.empty;;
let m = Map.insert "one" 1 m;;
let m = Map.insert "two" 2 m;;
let m = Map.insert "three" 3 m;;
Map.bindings m;;
```

```
[("one", 1); ("two", 2); ("three", 3)]
```

样例 3

```
let m = Map.empty;;  
let m = Map.insert "alpha" 42 m;;  
Map.lookup "beta" m;;
```

```
Exception: Not_found.
```

提示

你可以使用 `list` 来实现 `Map` 模块。模块的签名仅仅规定了模块向外暴露的项，你可以定义几个辅助函数。

练习 4 Pokemon Fight

目的

- 掌握 Functor 的用法。

题目描述

你需要模拟一个简单的宝可梦（Pokémon）战斗系统：

1. 定义 `Pokemon` 模块类型，其中包含属性：宝可梦的名称、生命值、攻击力、受到攻击时减少生命值：

```
module type Pokemon = sig、
  (* 名称 *)
  val name : string
  (* 生命值 *)
  val hp : int ref
  (* 攻击力 *)
  val attack : int
  (* 收到攻击时减少生命值，减少的数量应该是对方宝可梦的攻击力 *)
  val take_damage : int -> unit
end
```

2. 实现具体的宝可梦模块，它们应该符合 `Pokemon` 模块类型，比如：

```
module Pikachu : Pokemon = struct
  let name = "Pikachu"
  ...
end
```

3. 定义 `Battle` Functor，它接受两个符合 `Pokemon` 模块类型的模块，并提供战斗逻辑：

- `fight : unit -> unit` —— 两个宝可梦轮流攻击，直到其中一个生命值为 0。
- `print_winner : unit -> unit` —— 输出胜利者的名字。

```
module Battle : functor (P1 : Pokemon) (P2 : Pokemon) -> sig
  val fight : unit -> unit
  val print_winner : unit -> unit
end
```

样例

```
(* 假设你已经定义好了 Pikaku 模块和 Charmander 模块 *)
module PikachuVsCharmander = Battle (Pikachu) (Charmander)

let () = PikachuVsCharmander.fight(); PikachuVsCharmander.print_winner ();;
```

```
Pikachu wins!
```

练习 5 Task

目的

- 培养面向对象编程的思想和能力。
- 掌握 OCaml 中 Class 的用法。

题目描述

你需要设计一个任务管理系统，支持管理不同类型的任务（普通任务、定时任务、优先任务），并实现以下功能：

1. 任务类型：
 - 普通任务：仅包含标题和描述。
 - 定时任务：在普通任务的基础上还包含开始时间和结束时间。
 - 优先任务：在普通任务的基础上还包含优先级（如 1-5，数字越大优先级越高）。
2. 任务管理器：开发一个任务管理器管理所有任务。
2. 任务操作：
 - 添加任务：向任务管理器添加任务。
 - 列出任务：以字符串格式显示所有任务信息。
 - 排序任务：先按优先级排序（优先任务 > 其他任务）。再按开始时间排序（仅适用于定时任务）。
 - 状态管理：任务可以标记为完成或未完成。

提示

这道题目不对实现方式作出限制，你可以自由的设计类和继承关系。尝试运用封装、继承和多态。